MCMP117 · Week 13 · Course Notes

Fundamentals of Operating Systems

Processes & Job Control

Every command you type becomes a process. This week you learn to watch it, pause it, move it and stop it.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain how the shell turns every command into a process with `fork()`, `exec()` and `wait()`, and find its PID and PPID.
2. List and inspect processes with `ps` (`-f`, `-ef`, `aux`, `-o`), `pstree`, `top`/`htop` and `/proc/<pid>/status`, and read every column, including the load average.
3. Map the five process states of Week 3 to the STAT letters R, S, D, T and Z.
4. Run jobs in the foreground and background, and move them with &, Ctrl+Z, Ctrl+C, `jobs`, `fg` and `bg`.
5. Send signals with `kill`, `pkill` and `killall`; explain SIGTERM, SIGKILL, SIGINT, SIGHUP, SIGSTOP and SIGCONT, and the exit status $128 + n$.
6. Use `nohup`, `$$` and `$!`; change priorities with `nice` and `renice` and predict CPU shares (Week 5).
7. Create a zombie process on purpose, recognise it, and explain why only the parent can remove it.

Contents

1 From Week 12 to Week 13 1

2 Processes Revisited 2

3 Looking at Processes 2

4 Job Control 8

5 Signals 10

6 Priorities: `nice` and `renice` 15

7 Zombies and Orphans, Made Visible 16

8 Command Reference 18

9 Common Errors and What They Mean 18

10 Summary 19

11 Glossary 19

12 Try It Yourself 20

13 Review Questions 20

14 Looking Ahead: Week 14 21

15 Lab Sheet 22

16 Lab Sheet Solutions 23

1 From Week 12 to Week 13

Last week you connected programs with redirections and pipes. Behind every line such as `grep ERROR logs/system.log | wc -l` the shell quietly created *processes*: two of them, running at the same time, joined by a pipe. This week we look at those processes directly. Almost everything we need was introduced on paper in Week 3 (process, PID, process states, PCB, `fork()`/`exec()`/`wait()`, zombies) and in Week 5 (priority scheduling, nice values). Now you will see each of these ideas in a real Linux terminal and learn to control processes yourself.

About the outputs in these notes. Every session shown here is real output, captured on our lab machine as the ordinary user `student` in a freshly created `~/lab` folder. The commands are in the files `code/demo_*.sh` and the captured outputs in `code/out_*.txt`. Job control (Ctrl+Z, `fg`, `bg`) only works in an *interactive* shell that owns a terminal, so the demos were typed into a real terminal by a small helper program (`code/record_session.py`); read the `demo` files as "type these lines one by one". Lines starting with `#@` in those files are instructions such as "press Ctrl+Z".

On your machine, PIDs, times, dates, terminal names (`pts/0`) and user numbers will be different, and PIDs change with every run. Listings that contain all processes of the machine (`ps -ef`, `ps aux`) have been *trimmed*: we show the first lines and the processes of `student` only, and cut very long lines with "…". The order of `ls` and `sort` output can depend on the *locale*; we used `LC_ALL=C.UTF-8`.

Two harmless test programs appear everywhere this week: `sleep N` waits N seconds and does nothing (it spends its life in the *waiting* state), and `yes` prints the letter `y` forever; with `yes > /dev/null` its output is thrown away and it simply keeps one CPU core busy (it is always *ready* or *running*).

2 Processes Revisited

Recall from Week 3: a **program** is a passive file on disk, such as `/usr/bin/sleep`; a **process** is a program in execution. Each process has a unique **process identifier** (PID), knows the PID of its parent (PPID), and is described inside the kernel by its **process control block** (PCB). Processes form a tree, because each one (except the first) is created by another.

When you type a command, the shell (`bash`) does exactly what the Week 3 C program `fork_exec_wait.c` did:

1. `fork()` creates a copy of the shell, the child;
2. the child calls `exec()`, which replaces the copy of `bash` by the program you asked for; the PID stays the same;
3. the parent shell calls `wait()` and prints the next prompt only when the child has finished.

If the command ends with &, step 3 is skipped: the shell does not wait, prints the prompt immediately, and the command runs in the *background*. That is the entire difference between a foreground and a background command.

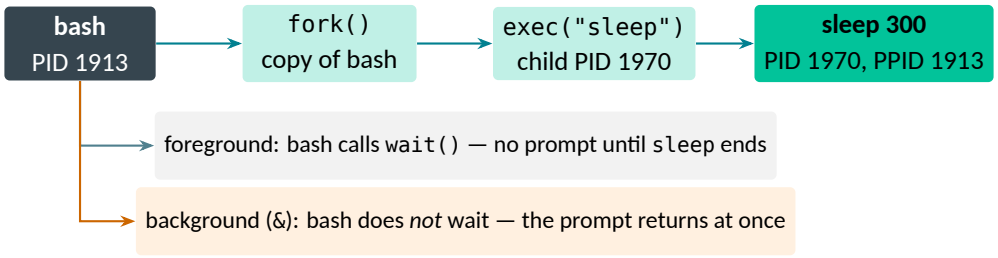


Figure 1: How the shell runs `sleep 300 &`. The PIDs are those of the real session in Section 3.

3 Looking at Processes

3.1 `ps`: a snapshot of your processes

`ps` (*process status*) prints a one-time snapshot of processes. Without options it shows only the processes attached to your current terminal. `$$` is a special shell variable that holds the PID of the shell itself.

```
student@okan-lab:~/lab$ echo $$
1913
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1969 pts/0    00:00:00 ps
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ yes > /dev/null &
[2] 1971
student@okan-lab:~/lab$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   1913  1912  0 16:58 pts/0    00:00:00 -bash
student   1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student   1971  1913 96 16:58 pts/0    00:00:00 yes
student   1973  1913  0 16:58 pts/0    00:00:00 ps -f
```

What to observe:

- Our shell has PID 1913; it appears in the list as `bash`, and `ps -f` shows its full name `-bash`. The leading dash marks a *login shell*.
- `ps` lists itself: when it takes its snapshot, it is a running process like any other. (After the two background commands, a second plain `ps`, omitted here, lists `bash`, `sleep`, `yes` and `ps`.)
- The two background commands print `[1] 1970` and `[2] 1971`: a *job number* in brackets and the PID.
- `ps -f` (*full format*) adds the owner (UID), the PPID, the CPU usage C, the start time and the whole command line. The PPID of `sleep` and `yes` is 1913: the shell is their parent. The PPID of the shell, 1912, belongs to the terminal program that started it.
- The column C of `yes` is 96: it uses almost all of one CPU core. PIDs jump (from 1913 to 1969) because other users created processes in the meantime.

3.2 Choosing columns with `-o`, and `pstree`

The option `-o` lets you choose exactly the columns you need. This is the form we use most often:

```
student@okan-lab:~/lab$ ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
  PID  PPID STAT  NI %CPU %MEM CMD
 1913  1912 Ss     0  0.5  0.0 -bash
 1970  1913 S      0  0.0  0.0 sleep 300
 1971  1913 R      0 97.4  0.0 yes
 1974  1913 R+     0  0.0  0.0 ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

STAT is the process state (Section 3.5), `NI` the nice value (Section 6). `yes` is in state R with 97.4% CPU; `sleep` is in state S with 0%. `pstree -p $$` draws the process tree of Week 3, starting at our shell, with real PIDs: the shell has three children. At the end of the session we stopped both jobs:

```
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 300
[2]+  Terminated              yes > /dev/null
```

`%1` means "job number 1" (Section 4); the shell reports `Terminated` for each job once.

3.3 All processes: `ps -ef`, `ps aux`, `ps -u`

Two option styles exist for historical reasons. `ps -ef` (System V style, with a dash) and `ps aux` (BSD style, *without* a dash) both list *every* process of *every* user; `aux` adds memory columns. On our lab machine the full lists have about 70 lines, most of them belonging to `root`, so we show only the beginning of `ps -ef` and then select the user `student` with `grep` (`ps -f -u student` gives the same selection in the format of `ps -f`):

```
student@okan-lab:~/lab$ ps -ef | head -4
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 16:17 ?        00:00:02 /process_api --firecracker-init --addr
```

```
    0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-connections --listen-vsock-port 2024
     --log-vsock-port 5002
root         2     0  0 16:17 ?        00:00:00 [kthreadd]
root         3     2  0 16:17 ?        00:00:00 [pool_workqueue_release]
student@okan-lab:~/lab$ ps aux | head -1
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
student@okan-lab:~/lab$ ps aux | grep '^student'
student   1913  0.6  0.0   8436  7532 pts/0    Ss   16:58   0:00 -bash
student   1970  0.0  0.0   3132  1996 pts/0    S    16:58   0:00 sleep 300
student   1971 99.0  0.0   3132  1884 pts/0    R    16:58   0:02 yes
student   1981  0.0  0.0   8340  4328 pts/0    R+   16:58   0:00 ps aux
student   1982  0.0  0.0   4100  2200 pts/0    S+   16:58   0:00 grep ^student
```

- **PID 1** is the first user-space process, started by the kernel at boot; it adopts orphans (Week 3). On our lab machine it is a small container start-up program called `process_api` (its long command line wraps above); on a normal Ubuntu installation PID 1 is `systemd`.
- **PID 2**, `kthreadd`, is the parent of all kernel threads; their names are shown in square brackets.
- `ps aux | grep '^student'` finds the student's lines — and also the `grep` process itself, which was running while `ps` took its snapshot. `pgrep` (Section 5.5) avoids this surprise.

3.4 Reading the columns

Column	Meaning	Example (above)
`PID, PPID`	process ID and parent's process ID (Week 3)	`1970`, `1913`
`UID, USER`	owner of the process (Week 11)	`student`
`STAT, S`	state letter and flags (Section 3.5)	`S`, `R+`, `Ss`
`%CPU, C`	CPU time divided by elapsed time since start, in %	`99.0` for `yes`
`%MEM`	resident memory divided by total RAM, in %	`0.0`
`VSZ`	virtual memory size in KiB (Weeks 6–7)	`3132`
`RSS`	resident set size: the part in RAM, in KiB	`1884`
`TTY`	terminal of the process; `?` = none (daemons)	`pts/0`
`STIME, START`	start time	`16:58`
`TIME`	total CPU time used so far	`0:02`
`CMD, COMMAND`	command line (`[...]` = kernel thread)	`sleep 300`

VSZ and RSS connect to Weeks 6–7: `yes` has 3132 KiB of virtual address space but only 1884 KiB resident in RAM — demand paging at work. The %CPU of `ps` is an average over the *whole life* of the process; `top` measures the last few seconds instead. A multithreaded process (Week 4) can show more than 100%: 200% means two cores fully busy.

3.5 Process states in `ps`: the STAT letters

Week 3 described five states: *new*, *ready*, *running*, *waiting* and *terminated*. Linux reports a state letter for each process in the STAT column of `ps` (the S column of `top`), and it adds one state that the textbook model does not have: *stopped*.

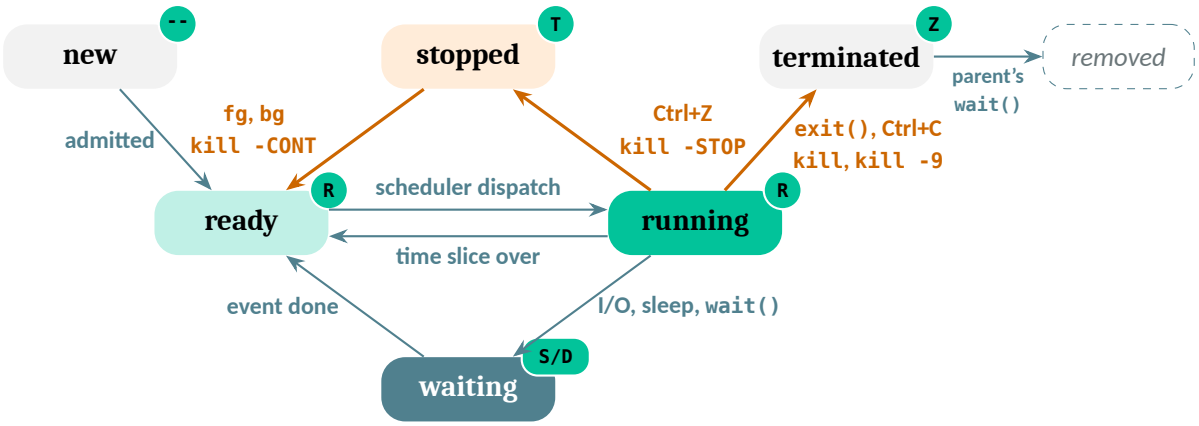


Figure 2: The Week 3 state diagram with the STAT letter of each state (mint badges) and the job-control actions of this week (orange arrows). Grey arrows are taken by the kernel and the scheduler.

Letter	Meaning	Week 3 state	Seen in these notes
R	running or runnable: on a CPU, or in the ready queue	running *or* ready	`yes > /dev/null`
S	interruptible sleep: waiting for an event (timer, keyboard, child)	waiting	`sleep`, `bash`
D	uninterruptible sleep, usually waiting for disk I/O	waiting	rare and short
T	stopped by Ctrl+Z or SIGSTOP	(Linux extra)	Section 4
Z	zombie: terminated, exit status not yet collected	terminated	Section 7

Characters after the letter are *flags*: `s` = session leader (your login shell shows `Ss`), `+` = member of the foreground process group (`ps` itself shows `R+`), `N` = low priority, i.e. positive nice value (`RN`, `SN`), `<` = high priority, `l` = multithreaded. Three remarks:

- Linux does not distinguish *ready* from *running*: both are R. On a machine with 2 cores at most 2 processes are really running at any instant; the other R processes are in the ready queue.
- The *new* state lasts only microseconds and has no letter.
- A process in state S can also be killed: `sleep` spends its whole life in S, and we kill it all the time. The diagram draws the `kill` arrow from *running* only to stay readable.

3.6 `/proc/<pid>/status`: the PCB as text

Since Week 3 we know that Linux shows parts of every PCB as text files under `/proc`. `$!` is the PID of the most recent background job:

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2118
student@okan-lab:~/lab$ echo $!
2118
student@okan-lab:~/lab$ grep -E '^(Name|State|Pid|PPid|Uid|Threads|VmRSS)' /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    2118
PPid:   2061
Uid:    1013    1013    1013    1013
VmRSS:      2004 kB
Threads:    1
student@okan-lab:~/lab$ grep ctxt /proc/$!/status
voluntary_ctxt_switches:    1
nonvoluntary_ctxt_switches: 2
student@okan-lab:~/lab$ grep -E '^(Name|State|PPid)' /proc/$$/status
Name:   bash
State:  S (sleeping)
PPid:   2060
```

The fields map onto the PCB of Week 3: `State` (S, waiting for its timer), `Pid` and `PPid` (identity and parent), `Uid` (the owner's user number; 1013 on our lab machine, typically 1000 on yours, Week 11), `VmRSS` (memory-management information), `Threads` (Week 4) and the context-switch counters (accounting information: a *voluntary* switch happens when the process gives up the CPU to wait, a *nonvoluntary* one when the scheduler takes the CPU away). (The full session in `code/out_proc.txt` also reads `/proc/$!/cmdline` and the links `cwd` and `exe`: the command line, the current directory and the program file.) Our own shell (`$$`) is also sleeping: it waits for the keyboard.

3.7 `top` and `htop`: the live view

`top` shows the processes sorted by CPU usage and refreshes every few seconds; press `q` to quit. To print it in these notes we used *batch mode* (`-b`) with one iteration (`-n 1`) and the user filter `-u student`, while a `yes` and a `sleep` (started with `&` half a minute earlier) were running:

```
student@okan-lab:~/lab$ cat /proc/loadavg
0.59 0.34 0.23 3/98 2262
student@okan-lab:~/lab$ nproc
2
student@okan-lab:~/lab$ top -b -n 1 -u student | head -15
top - 16:58:44 up 41 min,  0 user,  load average: 0.59, 0.34, 0.23
Tasks:  71 total,   2 running,  69 sleeping,   0 stopped,   0 zombie
%Cpu(s): 28.6 us, 23.8 sy,  0.0 ni, 42.9 id,  0.0 wa,  0.0 hi,  0.0 si,  4.8 st
MiB Mem :   8029.6 total,   7294.2 free,    598.8 used,    357.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   7430.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2260 student   20   0    3132   1876   1760 R 100.0   0.0   0:30.83 yes
 2204 student   20   0    8436   7528   3552 S   0.0   0.1   0:00.01 bash
 2261 student   20   0    3132   2004   1892 S   0.0   0.0   0:00.00 sleep
 2264 student   20   0    9380   5436   3308 R   0.0   0.1   0:00.00 top
 2265 student   20   0    3144   1956   1840 S   0.0   0.0   0:00.00 head
```

- **Line 1** (the same as the output of `uptime`): current time, time since boot, number of logged-in users (0 on our lab machine because nobody is logged in on its console) and the **load average** for the last 1, 5 and 15 minutes.

- **Line 2** counts all tasks of the machine by state: running (R), sleeping (S and D), stopped (T), zombie (Z).
- **Line 3** splits the CPU time of the *whole machine*: `us` user mode, `sy` kernel mode (dual mode, Week 2), `ni` niced user processes, `id` idle, wa waiting for I/O, `st` "stolen" by the virtualisation host. One busy core out of 2 is only $1/2 = 50\%$ of the machine.
- **Lines 4–5**: memory and swap in MiB (Weeks 6–7).
- **Process list**: PR is the priority, PR $= 20 +$ NI for normal processes; `VIRT`/`RES` are virtual and resident memory; S is the state; `%CPU` is measured over the last refresh interval. `yes` uses 100% of *one* core.

Useful keys in `top`: `P` sort by CPU, `M` sort by memory, `u` show one user, `k` kill (asks for a PID and a signal), `r` renice, `1` show every CPU, `q` quit. `htop` is a friendlier, coloured version: select a process with the arrow keys, F9 sends a signal, F7/F8 change the nice value, `F10` quits. Install it with `sudo apt install htop` if it is missing.

Reading the load average

The load average counts processes that are running or ready (state R), plus those in uninterruptible disk wait (D), averaged over 1, 5 and 15 minutes. It only makes sense relative to the number of cores c (`nproc` prints it; $c = 2$ on our lab machine):

load 0.59 : $0.59/2 = 29.5\%$ of the capacity is used,
load 2.00 : $2.00/2 = 100\%$: every core is busy, nobody waits,
load 3.00 : $3.00/2 = 150\%$: on average $3 - 2 = 1$ process waits in the ready queue.

If the 1-minute value is above the 15-minute value, the load is rising. Our snapshot was taken 30 s after `yes` started, so the 1-minute average (0.59) had not yet climbed to 1: the averages are exponentially decaying, not instant counts. In `/proc/loadavg`, the fourth field `3/98` means 3 runnable tasks now out of 98, and the last number is the most recently assigned PID. In Week 4 terms: a load above c means processes spend time in the ready queue, so their waiting time grows. (Numbers checked with `code/check_numbers.py`.)

4 Job Control

A **job** is one command line started by the shell: a single process, or several processes connected by pipes. The shell numbers its jobs `[1]`, `[2]`, … Only *one* job at a time owns the keyboard: the **foreground** job. The keys Ctrl+C and Ctrl+Z send signals (Section 5) to the foreground job only. Any number of jobs can run in the **background**.

In the **foreground** (`command`) the shell waits and shows no prompt until the job ends; the job reads the keyboard; Ctrl+C terminates it and Ctrl+Z stops it. Use it for short commands and interactive programs (`nano`, `less`, `top`). In the **background** (`command &`) the shell prints `[job] PID` and the prompt at once; keys do not reach the job (use `kill %n`); if it tries to read the keyboard it is stopped (`Stopped (tty input)`); its output still appears on the terminal unless you redirect it (Week 12). Use it for long, non-interactive work: compiling, copying, downloads.

4.1 Stopping a job with Ctrl+Z, continuing it with bg

```
student@okan-lab:~/lab$ sleep 100
^Z
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ jobs
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 T    sleep 100
 2402 R+   ps -o pid,stat,cmd
student@okan-lab:~/lab$ bg
[1]+ sleep 100 &
student@okan-lab:~/lab$ jobs -l
[1]+  2401 Running                 sleep 100 &
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 S    sleep 100
 2403 R+   ps -o pid,stat,cmd
```

1. `sleep 100` runs in the foreground: no prompt. Pressing **Ctrl+Z** (echoed as ^Z) stops it; the shell reports `[1]+ Stopped` and gives the prompt back.
2. `jobs` lists the jobs of this shell; `ps` shows the state T (stopped). A stopped process keeps its memory and open files but receives no CPU time: in Week 3 terms it is neither ready nor waiting, which is why Linux needs the extra state.
3. `bg` lets the current job continue in the *background*; the shell prints the command with an & added. `jobs -l` adds the PIDs. `sleep` is back in state S: it is waiting for its timer again, not stuck.

4.2 Starting with &, fg, Ctrl+C and `kill %n`

```
student@okan-lab:~/lab$ sleep 200 &
[2] 2404
student@okan-lab:~/lab$ yes > /dev/null &
[3] 2405
student@okan-lab:~/lab$ jobs
[1]   Running                 sleep 100 &
[2]-  Running                 sleep 200 &
[3]+  Running                 yes > /dev/null &
student@okan-lab:~/lab$ fg %3
yes > /dev/null
^C
student@okan-lab:~/lab$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 100
[2]+  Terminated              sleep 200
student@okan-lab:~/lab$ jobs
```

- `[3] 2405`: job number 3, PID 2405. In the `jobs` list, + marks the *current* job (the default for `fg` and `bg` without an argument) and `-` the previous one.
- `fg %3` brings `yes` to the foreground (the shell prints its command line); **Ctrl+C** (^C) terminates it.

- `kill %1 %2` sends SIGTERM to jobs 1 and 2. The shell reports `Terminated` once, just before the next prompt; the second `jobs` prints nothing.

4.3 Naming jobs and processes; `$$` and `$!`

You write	It means	Example
`%1, %2, …`	job number n of this shell	`fg %2, kill %1`
`%+` or `%%`	the current job (marked +)	`fg`
`%-`	the previous job (marked -)	`bg %-`
`%sleep`	the job whose command starts with `sleep`	`kill %sleep`
`1970`	a PID: valid in every terminal and for every tool	`kill 1970`
`$$`	PID of the current shell	`echo $$`
`$!`	PID of the most recent background job	`kill $!`

Job numbers exist only inside the shell that started the jobs: a second terminal has its own `[1]`. PIDs are global. **Careful:** `kill 1` (without `%`) means PID 1, not job 1 — see the common errors in Section 9. The variables `$$` and `$!` will be useful in Week 14 scripts.

5 Signals

5.1 What a signal is

A **signal** is a small, numbered notification that the kernel delivers to a process. It is the software counterpart of the hardware interrupt of Week 2: an interrupt makes the CPU stop what it is doing and run a handler; a signal makes one *process* stop what it is doing and react. Signals are sent by users (keys, the `kill` command), by other processes, and by the kernel itself (for example SIGCHLD when a child terminates, SIGSEGV after an illegal memory access).

The kernel marks the signal as pending and delivers it to the target process, which reacts in one of three ways: the *default action* of that signal (terminate, stop, continue, or ignore), a *handler* function of its own (*catching* the signal), or ignoring it. Two signals are special: **SIGKILL (9)** and **SIGSTOP (19)** can be neither caught nor ignored — the kernel acts alone. Ordinary users may signal only their own processes; `root` may signal any (a protection rule, Week 11).

> **Why "kill" and "hang up"?**
> In early UNIX the command that sends signals was named `kill` because the default action of most signals is to terminate the process; the name stayed even though `kill -STOP` or `kill -CONT` kill nothing. SIGHUP ("hang up") comes from the time when terminals were connected through telephone lines and modems: when the line was hung up, the processes of that session received SIGHUP. Today closing a terminal window or an SSH connection does the same.

5.2 The signal table

`kill -l` lists all 64 signal numbers of Linux (first lines below; the full list is in `code/out_signals.txt`). `kill -l 15` converts a number to a name, `kill -l KILL` a name to a number.

```
student@okan-lab:~/lab$ kill -l
 1) SIGHUP   2) SIGINT   3) SIGQUIT  4) SIGILL   5) SIGTRAP
 6) SIGABRT  7) SIGBUS   8) SIGFPE   9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
```

No.	Name	Sent by	Default action	Catchable?
1	SIGHUP	closing the terminal or SSH session	terminate	yes
2	SIGINT	Ctrl+C	terminate	yes
3	SIGQUIT	Ctrl+\	terminate + core dump	yes
9	SIGKILL	`kill -9`: the last resort	terminate at once	**no**
11	SIGSEGV	the kernel, after an illegal memory access (Week 7)	terminate + core dump	yes
15	SIGTERM	`kill` (default), `pkill`, `killall`, system shutdown	terminate	yes
17	SIGCHLD	the kernel, when a child terminates or stops	ignore	yes
18	SIGCONT	`fg`, `bg`, `kill -CONT`	continue if stopped	yes
19	SIGSTOP	`kill -STOP`	stop	**no**
20	SIGTSTP	Ctrl+Z	stop	yes

The numbers are those of Linux on x86 and ARM; some differ on other UNIX systems, so names are safer than numbers (except 9 and 15, which are the same everywhere). You may write the name with or without the prefix: `kill -TERM`, `kill -SIGTERM` and `kill -15` are equivalent.

5.3 SIGTERM, SIGKILL, SIGINT and the exit status

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2541
student@okan-lab:~/lab$ kill $!
student@okan-lab:~/lab$ wait $!
[1]+  Terminated              sleep 300
student@okan-lab:~/lab$ echo $?
143
student@okan-lab:~/lab$ sleep 300 &
[1] 2542
student@okan-lab:~/lab$ kill -9 $!
student@okan-lab:~/lab$ wait $!
[1]+  Killed                  sleep 300
student@okan-lab:~/lab$ echo $?
137
```

The built-in command `wait $!` waits for the background job to end and makes the shell report *how* it ended: `Terminated` (SIGTERM), `Killed` (SIGKILL) or `Interrupt` (SIGINT: a third experiment with `kill`

-INT, in the full output file, prints `Interrupt`). Then `$?`, the exit status of the last command, encodes the signal:

Exit status of a process killed by a signal

If a process is terminated by signal number n, the shell sets its exit status to $128 + n$:

$$\text{SIGTERM: } 128 + 15 = 143, \qquad \text{SIGKILL: } 128 + 9 = 137, \qquad \text{SIGINT: } 128 + 2 = 130.$$

A normal exit gives a status between 0 and 127 (usually 0 for success). Conversely, a status $s > 128$ means "killed by signal $s - 128$".

5.4 Pausing and resuming: SIGSTOP and SIGCONT

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2544
student@okan-lab:~/lab$ kill -STOP $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 T    sleep 300

[1]+  Stopped                 sleep 300
student@okan-lab:~/lab$ kill -CONT $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 S    sleep 300
student@okan-lab:~/lab$ kill $!
```

`kill -STOP` puts `sleep` into state `T` — exactly the state that Ctrl+Z produces, which is why the shell reports the job as `Stopped`. `kill -CONT` lets it continue (`S` again). This is what `fg` and `bg` do internally: they send SIGCONT.

5.5 Choosing processes by name: `pgrep`, `pkill`, `killall`

```
student@okan-lab:~/lab$ sleep 401 &
[2] 2548
[1]   Terminated              sleep 300
student@okan-lab:~/lab$ sleep 402 &
[3] 2549
student@okan-lab:~/lab$ yes > /dev/null &
[4] 2550
student@okan-lab:~/lab$ pgrep -u student -l sleep
2548 sleep
2549 sleep
student@okan-lab:~/lab$ pkill -u student sleep
[2]   Terminated              sleep 401
[3]-  Terminated              sleep 402
student@okan-lab:~/lab$ killall -u student yes
[4]+  Terminated              yes > /dev/null
student@okan-lab:~/lab$ jobs
student@okan-lab:~/lab$ pgrep -u student -l sleep
student@okan-lab:~/lab$ echo $?
1
```

`pgrep` prints the PIDs of processes whose name matches a pattern (`-l` adds the name, `-c` counts them); `pkill` sends them a signal (SIGTERM by default); `killall` does the same with an exact program name.

The last `pgrep` finds nothing, and its exit status is 1. **Always add `-u student`** (your own user name): without it these tools also try to signal other users' processes, which fails with "Operation not permitted" — and, run as root, would actually kill them. Test the pattern with `pgrep` before using `pkill`: `pkill -u student bash` would close all your terminals.

5.6 Catching signals: `catch.c`

A program can install its own *handler* for most signals. The program `code/catch.c` catches SIGINT and SIGTERM, and also tries — and fails — to install a handler for SIGKILL. The listing shows its core; the complete file adds the `#include` lines, the `struct sigaction` set-up and the start-up message:

```c
static volatile sig_atomic_t got = 0;
static void handler(int sig) { got = sig; }  /* tiny! */

sa.sa_handler = handler;
sigaction(SIGINT,  &sa, NULL);   /* Ctrl+C, 2: caught */
sigaction(SIGTERM, &sa, NULL);   /* kill,  15: caught */
if (sigaction(SIGKILL, &sa, NULL) < 0)  /* 9: refused */
    perror("catch: sigaction(SIGKILL)");

for (;;) {
    pause();                     /* sleep until a signal */
    if (got == SIGINT)
        printf("... ignoring it, still running\n");
    if (got == SIGTERM) {
        printf("... cleaning up, bye\n");
        return 0;
    }
}
```

Compiled with `gcc -Wall -O2` (no warnings) and run in `~/lab/src`:

```
student@okan-lab:~/lab$ cd src
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o catch catch.c
student@okan-lab:~/lab/src$ ./catch
catch: sigaction(SIGKILL): Invalid argument
catch: PID 2694, waiting for signals
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Z
[1]+  Stopped                 ./catch
student@okan-lab:~/lab/src$ bg
[1]+ ./catch &
student@okan-lab:~/lab/src$ kill -INT %1
student@okan-lab:~/lab/src$ catch: got SIGINT (2) -- ignoring it, still running
kill -TERM %1
student@okan-lab:~/lab/src$ catch: got SIGTERM (15) -- cleaning up, bye
jobs
[1]+  Done                    ./catch
student@okan-lab:~/lab/src$ ./catch &
[1] 2695
student@okan-lab:~/lab/src$ catch: sigaction(SIGKILL): Invalid argument
catch: PID 2695, waiting for signals
kill -9 %1
student@okan-lab:~/lab/src$ jobs
[1]+  Killed                  ./catch
```

The installation of a SIGKILL handler fails (`Invalid argument`): the kernel refuses it. Ctrl+C and `kill -INT` no longer terminate the program — the handler runs and the program carries on. Ctrl+Z still stops it (SIGTSTP is not caught) and `bg` continues it. SIGTERM makes it clean up and exit normally (`jobs` reports `Done`, not `Terminated`), and `kill -9` always works (`Killed`). Messages printed by a background job appear wherever the cursor is — right after the prompt — so the next command you type starts on a new line. This looks odd, but it is normal.

5.7 Stopping a process safely

Ask first, force last

1. Foreground job: press Ctrl+C (SIGINT).
2. Otherwise: `kill PID` (SIGTERM) and wait 5–10 seconds.
3. Check with `ps -p PID` whether it is still there, and in which state.
4. Only then: `kill -9 PID` (SIGKILL).

Warning about `kill -9`. SIGKILL gives the program no chance to save open documents, flush buffers or delete temporary and lock files; a database may need repair afterwards, and the program's children are not informed. And sometimes `kill -9` does not help at all: a zombie (Z) is already dead (Section 7); a process in state D receives the signal only when its disk I/O finishes; and another user's process cannot be signalled ("Operation not permitted").

5.8 SIGHUP and nohup

When you close a terminal (or an SSH connection), the shell's jobs receive SIGHUP and, by default, terminate. `nohup command` starts a command that ignores SIGHUP. In this session we simulate closing the terminal with `kill -HUP`:

```
student@okan-lab:~/lab$ echo "my shell is PID $$"
my shell is PID 2775
student@okan-lab:~/lab$ sleep 500 &
[1] 2831
student@okan-lab:~/lab$ echo "last background job is PID $!"
last background job is PID 2831
student@okan-lab:~/lab$ nohup sleep 600 &
[2] 2832
student@okan-lab:~/lab$ nohup: ignoring input and appending output to 'nohup.out'
nohup sleep 700 > /dev/null 2>&1 &
[3] 2834
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  nohup.out  photos  projects  scripts  src
student@okan-lab:~/lab$ jobs -l
[1]   2831 Running                 sleep 500 &
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill -HUP %1 %2 %3
student@okan-lab:~/lab$ jobs -l
[1]   2831 Hangup                  sleep 500
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill %2 %3
```

The plain `sleep 500` ended (Hangup), the two `nohup` jobs kept running. If the output of a `nohup` command goes to the terminal, `nohup` redirects it to the file `nohup.out` (and says so: the message appeared after the prompt because the background `nohup` printed it a moment later). The clean form is

```
nohup long_command > run.log 2>&1 &
```

which uses the redirections of Week 12. Typical use: start a long job on a remote server over SSH, log out, and look at `run.log` later.

6 Priorities: `nice` and `renice`

In Week 5 we met priority scheduling and the Linux scheduler, which shares the CPU among runnable processes in proportion to a *weight*. The weight comes from the **nice value**, an integer from -20 to $+19$ (40 values, default 0). As in Week 5, a *smaller* number means a *higher* priority: a process with a high nice value is "nice" to the others. `top` shows the priority PR $= 20 +$ NI.

```
student@okan-lab:~/lab$ nice
0
student@okan-lab:~/lab$ nice -n 10 yes > /dev/null &
[1] 2971
student@okan-lab:~/lab$ ps -o pid,ni,stat,cmd
  PID  NI STAT CMD
 2914   0 Ss   -bash
 2971  10 RN   yes
 2972   0 R+   ps -o pid,ni,stat,cmd
student@okan-lab:~/lab$ renice -n 15 -p $!
2971 (process ID) old priority 10, new priority 15
student@okan-lab:~/lab$ renice -n 5 -p $!
renice: failed to set priority for 2971 (process ID): Permission denied
student@okan-lab:~/lab$ nice -n -5 sleep 1
nice: cannot set niceness: Permission denied
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ taskset -c 1 yes > /dev/null &
[2] 2976
[1]   Terminated              nice -n 10 yes > /dev/null
student@okan-lab:~/lab$ taskset -c 1 nice -n 10 yes > /dev/null &
[3] 2977
student@okan-lab:~/lab$ ps -o pid,ni,stat,%cpu,cmd
  PID  NI STAT %CPU CMD
 2914   0 Ss    0.0 -bash
 2976   0 R    89.3 yes
 2977  10 RN    9.6 yes
 2978   0 R+    0.0 ps -o pid,ni,stat,%cpu,cmd
student@okan-lab:~/lab$ kill %2 %3
```

- `nice` without arguments prints the current nice value, 0. `nice -n 10 command` starts a command with nice value 10; `ps` shows `NI 10` and the flag `N` (`RN`).
- `renice -n 15 -p PID` changes the value of a running process. `renice` calls it "priority" in its message; it is the nice value. Raising it ($10 \to 15$) is allowed; lowering it again ($15 \to 5$) is `Permission denied` for ordinary users — otherwise everybody would set -20. `nice -n -5` prints a warning and runs the command at nice 0.
- The last experiment repeats the Week 5 measurement with `yes`: two CPU-bound processes are pinned to the same core with `taskset -c 1` (without pinning, each would get its own core and

both would show about 100%).

Predicting the CPU shares

The kernel's weight for nice 0 is 1024, for nice 10 it is 110 (each nice step changes the weight by a factor of about 1.25, and $1.25^{10} \approx 9.31$). Two CPU-bound processes on one core get

$$\frac{1024}{1024+110} \approx 90.3\,\%, \qquad \frac{110}{1024+110} \approx 9.7\,\%, \qquad \text{ratio } \frac{1024}{110} \approx 9.31.$$

We measured 89.3% and 9.6%, a ratio of $89.3/9.6 \approx 9.30$. (The two values do not add up to exactly 100% because `ps` averages over each process's lifetime and the two did not start at exactly the same moment. Numbers checked with `code/check_numbers.py`.)

As `root`, `sudo renice -n -5 -p PID` would work (Week 11) — but on a shared machine, do not give your own processes priority over everyone else's.

7 Zombies and Orphans, Made Visible

Week 3 defined a **zombie** as a process that has terminated but whose parent has not yet collected its exit status with `wait()`. The kernel has already freed its memory and closed its files; only the process-table entry with the PID and the exit status remains. In Week 3 we produced one with a shell trick; now we write the smallest possible C program that does it on purpose, `code/zombie.c`:

```c
pid_t pid = fork();                 /* one process becomes two */

if (pid == 0) {                     /* ---- child ---- */
    printf("child  %d: exiting now with status 7\n", getpid());
    exit(7);                        /* terminated -> zombie until reaped */
}
/* ---- parent: does NOT wait() for secs seconds ---- */
printf("parent %d: child is %d, sleeping %d s without wait()\n",
       getpid(), pid, secs);
sleep(secs);                        /* meanwhile the child is a zombie */

waitpid(pid, &status, 0);           /* collect the exit status: reap */
printf("parent %d: reaped child %d, exit status %d\n",
       getpid(), pid, WEXITSTATUS(status));
```

The listing shows the heart of the program; the complete file adds the `#include` lines, reads `secs` from the command line (default 30) and checks for a failed `fork()`. The child exits at once with status 7; the parent sleeps for `secs` seconds *without* calling `wait()`, then reaps the child with `waitpid()`. We copied `zombie.c` into `~/lab/src` and ran:

```
student@okan-lab:~/lab$ cd src
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o zombie zombie.c
student@okan-lab:~/lab/src$ ./zombie 25 &
[1] 3119
student@okan-lab:~/lab/src$ parent 3119: child is 3120, sleeping 25 s without wait()
child  3120: exiting now with status 7
ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
```

```
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3121  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ grep State /proc/$(pgrep -u student -n zombie)/status
State:  Z (zombie)
student@okan-lab:~/lab/src$ kill -9 $(pgrep -u student -n zombie)
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3125  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ wait
parent 3119: reaped child 3120, exit status 7
[1]+  Done                    ./zombie 25
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3128  3057 R+   ps -o pid,ppid,stat,cmd
```

- PID 3120, the child, is in state Z, its name is shown in brackets and marked `<defunct>`; its parent is 3119. `/proc` agrees: `State: Z (zombie)`. (`pgrep -n` selects the *newest* matching process, which is the child.)
- `kill -9` on the zombie does *nothing*: you cannot kill what is already dead.
- The shell's `wait` waits for the background job `./zombie`; after 25 seconds the parent calls `waitpid()`, receives the exit status 7, and the zombie disappears from the list.

A few zombies are harmless. A buggy parent that creates children in a loop and never waits for them can, however, fill the process table: PIDs are limited (`/proc/sys/kernel/pid_max` is 32768 on our machine, so the largest PID is 32767), and then no new process can be created. The cure is to fix or end the *parent*: when the parent terminates, the zombie becomes an **orphan**, is adopted by PID 1 (or a "subreaper" process), and that new parent reaps it at once (Week 3). The kernel helps parents with the signal SIGCHLD, sent when a child terminates; a well-written parent reacts to it by calling `wait()`.

8 Command Reference

Inspecting		Job control	
`ps / ps -f`	this terminal (short/full)	`cmd &`	start in the background
`ps -ef / ps aux`	all processes	`Ctrl+C / Ctrl+Z`	SIGINT / SIGTSTP to the foreground job
`ps -u student`	one user's processes	`jobs / jobs -l`	list jobs (with PIDs)
`ps -o pid,stat,cmd`	choose the columns	`fg %n / bg %n`	to the foreground / continue in the background
`ps -p PID`	one process	`wait [%n]`	wait for a background job
`ps --sort=-%cpu`	highest CPU first		
Inspecting (cont.)		**Signals**	
`pstree -p [PID]`	process tree with PIDs	`kill PID / kill %n`	SIGTERM (15)
`pgrep -u student -l name`	PIDs and names (`-c` count, `-n` newest)	`kill -INT / -HUP / -9`	SIGINT, SIGHUP, SIGKILL
`top / htop`	live view (q quits)	`kill -STOP / -CONT`	pause / resume
`uptime / nproc`	load average / cores	`kill -l [n]`	list signals / number to name
`cat /proc/PID/status`	the PCB as text	`pkill -u student pat`	signal by name pattern
`echo $$ / echo $!`	PID of shell / last background job	`killall -u student name`	signal by exact name
Priority		**Surviving logout**	
`nice -n 10 cmd`	start with nice value 10	`nohup cmd > log 2>&1 &`	ignore SIGHUP, output to `log`
`renice -n 15 -p PID`	change it (users: only upwards)	`taskset -c 1 cmd`	run on core 1 only (experiments)

9 Common Errors and What They Mean

`-bash: kill: (1) - Operation not permitted` You typed `kill 1` and meant job 1. Without `%` the number is a PID, and PID 1 belongs to root. Use `kill %1`. (Real output, Lab Sheet solution B4.)

`-bash: kill: %6: no such job` There is no job 6 in *this* shell. Check with `jobs`; jobs from another terminal must be addressed by PID.

`renice: failed to set priority for ... Permission denied` An ordinary user tried to lower a nice value. Only raising is allowed.

`nice: cannot set niceness: Permission denied` A negative nice value as an ordinary user. The command runs anyway, at nice 0.

`pkill: killing pid ... failed: Operation not permitted` The pattern also matched another user's process. Add `-u` with your user name.

The prompt does not come back A foreground job is running (you forgot the &). Press Ctrl+C to end it, or

Ctrl+Z and then bg to keep it running in the background.

`[1]+ Stopped (tty input)` A background job wanted to read the keyboard (try `nano &` or `cat &`). Bring it to the foreground with `fg`.

`There are stopped jobs.` You typed `exit` while jobs are stopped. Resume or kill them (`jobs`, `fg`, `kill %n`); typing `exit` a second time exits anyway, and the stopped jobs are terminated.

A `Z <defunct>` line survives `kill -9` A zombie is already dead. Only a `wait()` by its parent removes it; end or fix the parent.

`grep` appears in its own result A pipeline such as `ps aux | grep sleep` also lists the `grep` process itself. Use `pgrep -l sleep` instead.

10 Summary

Week 13 in eight sentences

1. Every command is a process created by the shell with `fork()` and `exec()`; in the foreground the shell `wait()`s, with & it does not.
2. `ps` (`-f`, `-ef`, `aux`, `-o`) and `pstree` take snapshots; `top`/`htop` give a live view; `/proc/<pid>/status` shows the PCB.
3. STAT letters: R = ready or running, S/D = waiting, T = stopped (a Linux extra), Z = terminated but not yet reaped.
4. The load average counts processes that want a CPU; compare it with the number of cores.
5. Ctrl+Z stops the foreground job, `bg` and `fg` continue it, `jobs` lists jobs; `%n` names a job, a plain number a PID.
6. `kill` sends signals (default SIGTERM, 15); Ctrl+C sends SIGINT (2); SIGKILL (9) and SIGSTOP (19) cannot be caught; a process killed by signal n has exit status $128 + n$.
7. `nohup` protects a job from SIGHUP; `nice`/`renice` set the nice value $-20 \ldots 19$, which users may only raise; nice 10 against nice 0 gets about $1/9.3$ of the CPU.
8. A zombie cannot be killed; only its parent's `wait()` (or the parent's death and adoption by PID 1) removes it.

11 Glossary

Background job A job started with &; the shell does not wait for it.

Foreground job The one job that receives keyboard input and the signals of Ctrl+C and Ctrl+Z.

Job A command line started by the shell (one or more processes), numbered `[1]`, `[2]`, …

Load average Average number of processes running, ready or in disk wait over 1, 5 and 15 minutes.

Nice value Number from -20 to 19 that sets a process's CPU weight; higher is "nicer" (lower priority).

Orphan A process whose parent has terminated; adopted by PID 1.

Signal Numbered software notification delivered by the kernel to a process.

SIGKILL / SIGTERM Signal 9 (cannot be caught, immediate termination) / signal 15 (polite request to terminate).

Stopped (T) State of a process paused by SIGTSTP or SIGSTOP; continued by SIGCONT.

Zombie (Z) A terminated process whose exit status has not yet been collected by its parent.

12 Try It Yourself

Processes on your own machine

Linux (lab PCs, WSL, or a virtual machine), in `~/lab`; compile the two programs with `gcc -Wall -O2 -o zombie zombie.c` and `gcc -Wall -O2 -o catch catch.c`.

1. Start `top` in one terminal and `yes > /dev/null &` three times in another. Watch the load average climb over a minute. Which value does it approach?
2. Run `nano notes.txt`, press Ctrl+Z, run `ls`, then `fg`. Your text is still there.
3. Change `catch.c` so that it also catches SIGTSTP. What happens now when you press Ctrl+Z?
4. Run `./zombie 60 &`, find the zombie, then kill the *parent* with `kill PID`. What happens to the zombie, and why?

13 Review Questions

Conceptual

1. What does the shell do differently for `sleep 10` and `sleep 10 &`? Use the words `fork()`, `exec()` and `wait()`.
2. Map each of the five Week 3 states to the letters that `ps` shows. Which Linux state has no Week 3 counterpart?
3. What is the difference between `kill 3` and `kill %3`?
4. Compare SIGTERM and SIGKILL. Why should SIGTERM be tried first? In which situations does even SIGKILL not remove a process from the list?
5. What do `fg` and `bg` send to a stopped job, and what is the difference between them?
6. Why may an ordinary user raise but not lower a nice value?

Numerical

7. A job ended and `echo $?` prints 130. What happened? What would 129 and 137 mean?
8. A machine with 4 cores shows `load average: 6.00, 3.00, 1.00`. Interpret each value. Is the load rising or falling?

9. Two CPU-bound processes share one core; one has nice 0 (weight 1024), the other nice 5 (weight 335). Compute their CPU shares.
10. A process in `top` shows `NI 19`. What is its PR? What is the PR of a process with nice -20?
11. A shell has four jobs. The commands `sleep 50 &`, `sleep 60 &`, `sleep 70` (then Ctrl+Z) were typed in this order after `yes > /dev/null &`. Which job number does each have, and what is the current job?

Answers to the numerical questions

7. $130 = 128 + 2$: terminated by SIGINT (Ctrl+C). $129 = 128 + 1$: SIGHUP (hang-up). $137 = 128 + 9$: SIGKILL.
8. Relative to 4 cores: last minute $6/4 = 150\%$ (on average $6 - 4 = 2$ processes waiting for a CPU); last 5 minutes $3/4 = 75\%$; last 15 minutes $1/4 = 25\%$. The 1-minute value is the largest: the load is rising.
9. $1024/(1024 + 335) = 1024/1359 \approx 75.3\%$ and $335/1359 \approx 24.7\%$; ratio $1024/335 \approx 3.06 \approx 1.25^5 \approx 3.05$.
10. PR $= 20 + 19 = 39$; for nice -20: PR $= 20 - 20 = 0$.
11. `yes` is `[1]`, `sleep 50` `[2]`, `sleep 60` `[3]`, `sleep 70` `[4]` (stopped). The current job (+) is the most recently stopped or started one, `[4]`; `[3]` is marked `-`.

14 Looking Ahead: Week 14

Next week we put commands into files and turn them into programs: **Introduction to Shell Scripting**. You will write scripts with a shebang line, make them executable with `chmod +x`, and use variables, quoting, command substitution `$( )`, arithmetic `$(( ))`, positional parameters, `read`, the exit status `$?`, tests, `if`, loops and functions. This week's material returns at once: `$$` and `$!` to manage background jobs, `wait` to wait for them, and the $128 + n$ rule when a script checks why a command failed. Week 15 is the final exam review.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapter 3 (Processes), in particular Sections 3.1–3.3 (process concept, scheduling, operations on processes), and Section 5.7 on Linux scheduling (nice values).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 4, "The Abstraction: The Process": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Chapter 5, "Interlude: Process API" (fork, exec, wait, signals, `ps`, `top`, `kill`): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
- The GNU Bash manual, section "Job Control": `https://www.gnu.org/software/bash/manual/html_node/Job-Control.html`

15 Lab Sheet

Preparation: `bash mcmp117_lab_setup.sh` then `cd ~/lab`. Copy `zombie.c` from the Week 13 `code/` folder into `~/lab/src`. Use only `sleep` and `yes` as test processes and replace `student` by your own user name. For every exercise, write down the command line(s) and the relevant output, and answer the question. Points in brackets, 100 in total. Solutions are in Section 16; try first!

Part A --- Inspecting processes (26 points)

A1 [4] Print the PID of your shell and find the same number in the output of `ps`.

A2 [5] Run `ps -f`. Which PPID does the `ps` process have, and why is it equal to `$$`?

A3 [6] Start `sleep 1000 &`. Using `/proc/$!/status`, report its `State` and `PPid`. Which Week 3 state is this?

A4 [5] Start two more `sleep` jobs and draw the tree that `pstree -p $$` prints.

A5 [6] Start `yes > /dev/null &` and show `pid`, `stat` and `cmd` of all processes of this terminal. Explain why `yes` is R and `sleep` is S.

Part B --- Job control (24 points)

B1 [6] Run `sleep 500` in the foreground and stop it with Ctrl+Z. Show it with `jobs` and show its STAT with `ps`.

B2 [6] Continue the job of B1 in the background. Prove with `jobs -l` and `ps` that it runs again.

B3 [6] Start three `sleep` jobs in the background. Bring job 2 to the foreground and end it with Ctrl+C. What does `jobs` show now?

B4 [6] Type `kill 1`, then `kill %1`. Explain the two different results.

Part C --- Signals (26 points)

C1 [8] End three `sleep 300 &` jobs with SIGTERM, SIGKILL and SIGINT respectively. After each one, run `wait $!` and `echo $?`. Explain the three numbers.

C2 [6] Pause a background `sleep` with `kill -STOP` and resume it with `kill -CONT`; show STAT after each step.

C3 [6] Start `sleep 901`, `sleep 902` and `sleep 903` in the background. Count your `sleep` processes with `pgrep`, terminate all of them with one command, and count again.

C4 [6] Start `sleep 800 &` and `nohup sleep 900 > /dev/null 2>&1 &`. Send SIGHUP to both jobs. Which one survives, and why?

Part D --- Priority, zombies and challenges (24 points)

D1 [7] Start `sleep 600` with nice value 15 and check `NI` and `STAT`. Raise its nice value to 19, then try to set it back to 0. Explain the result.

D2 [8] Compile `zombie.c` with `gcc -Wall -O2`, run `./zombie 30 &` and show the zombie with `ps`. Try `kill -9` on it. What happens, and what finally removes it?

D3 [4] *Challenge.* With one command line, show your three processes with the highest `%CPU` (plus the header line).

D4 [5] *Challenge (Week 12).* With one pipeline, count how many of your processes are in each state (first letter of `STAT`).

16 Lab Sheet Solutions

The solutions were run in a freshly created lab. Outputs that already appeared in the tutorial sections are referenced; the others (A3, A5, B4, C3, D1, D3, D4) come from `code/demo_lab.sh`, whose complete real output is in `code/out_lab.txt`. Your PIDs will differ. Other correct solutions exist; what counts is the right result and a sensible explanation.

	Command line(s)	Expected output / answer
A1	`echo $$` `ps`	A number such as 1913; the line with that PID in `ps` is `bash` (Section 3).
A2	`ps -f`	The PPID of `ps -f` equals the shell's PID: the shell forked the process that became `ps`.
A3	`sleep 1000 &` `grep -E '^(State|PPid)' /proc/$!/status`	`State: S (sleeping)`, `PPid:` the shell's PID (3207 in `out_lab.txt`). Week 3 state: *waiting* (for a timer).
A4	`sleep 2000 & sleep 3000 &` `pstree -p $$`	`bash(PID)` with the children `pstree`, and three `sleep` processes, as in Section 3.
A5	`yes > /dev/null &` `ps -o pid,stat,cmd`	`yes` is R: it always wants the CPU (ready or running). `sleep` is S: it waits for its timer. `ps` itself is R+, `-bash` Ss.

B1	`sleep 500`, then Ctrl+Z
	`jobs`
	`ps -o pid,stat,cmd`
		`[1]+ Stopped sleep 500`; `STAT` T (Section 4).

B2	`bg`
	`jobs -l`
	`ps -o pid,stat,cmd`
		`[1]+ sleep 500 &`, then `Running`; `STAT` S again.

B3	`sleep 100 & sleep 200 & sleep 300 &`
	`fg %2`, then Ctrl+C
	`jobs`
		`jobs` lists only jobs 1 and 3 as `Running`; job 2 has ended (no `Terminated` message: it ended in the foreground).

B4	`kill 1` `kill %1`	`-bash: kill: (1) - Operation not permitted`: `kill 1` addresses PID 1, owned by root. `kill %1` terminates job 1 (`Terminated` at the next prompt).

C1	`sleep 300 &`
	`kill $!` (then `-9`, `-INT`)
	`wait $!; echo $?`
		143, 137, 130 = $128+15$, $128+9$, $128+2$ (Section 5).

C2	`sleep 300 &`
	`kill -STOP $!; ps -o pid,stat,cmd -p $!`
	`kill -CONT $!; ps -o pid,stat,cmd -p $!`
		T, then S.

C3	`pgrep -c -u student sleep`
	`pkill -u student sleep`
	`pgrep -c -u student sleep`
		3, three `Terminated` messages, then `0`. (If jobs from earlier exercises are still running, the first count is larger.)

C4	`sleep 800 &`
	`nohup sleep 900 > /dev/null 2>&1 &`
	`kill -HUP %1 %2; jobs`
		`sleep 800`: Hangup. The `nohup` job keeps `Running`: `nohup` set SIGHUP to "ignore" before exec'ing `sleep` (Section 5.8).

	Command line(s)	Expected output / answer

D1	`nice -n 15 sleep 600 &`
	`ps -o pid,ni,stat,cmd -p $!`
	`renice -n 19 -p $!`
	`renice -n 0 -p $!`
		`NI` 15, `STAT` `SN`; `old priority 15, new priority 19`; then `Permission denied`: users may only raise the nice value.

D2	`gcc -Wall -O2 -o zombie zombie.c`
	`./zombie 30 &`
	`ps -o pid,ppid,stat,cmd`
	`kill -9 <child PID>`
		The child shows Z and `[zombie] <defunct>`. `kill -9` changes nothing: it is already dead. After 30 s the parent's `waitpid()` reaps it (Section 7); killing the parent would also remove it (adoption by PID 1).

D3	`ps -u student -o pid,stat,%cpu,cmd --sort=-%cpu | head -4`	Header plus three lines; `yes` first with about 98% (`head -4` = header + 3).
D4	`ps -u student -o stat= | cut -c1 | sort | uniq -c`	In our run `2 R` (`yes`, `ps`) and `5 S` (`-bash`, `sleep 600`, `cut`, `sort`, `uniq`). `stat=` prints the column without a header; `cut -c1` keeps the state letter; `sort | uniq -c` counts (Week 12). The pipeline's own processes are counted too.